

Context-Adaptive Multi-Critic Controller

using Deep Learning-Based Cost Weighting in Autonomous Navigation

Pravin Oli

Internal supervisor: **Prof. Zoltán Istenes** (ELTE)

External supervisors: **R. Tommasi & Dr. D. Horváth** (European Knowledge Centre Ltd.)

Contents

- 1. INTRODUCTION** — The Local Navigation Task
- 2. LITERATURE REVIEW** — Classical vs Learned vs Hybrid
- 3. OBJECTIVES** — Objectives & Research Questions
- 4. METHODOLOGY**
 - A: Overview
 - B: Perception
 - C: Network
 - D: MPPI Rollout
 - E: Algorithm & Critics

- 5. EXPERIMENTAL DESIGN**
- 6. TRAINING CONVERGENCE**
- 7. EVALUATION METRICS**
- 8. RESULTS**
 - A: Adaptation Signature
 - B: Tour & Findings
 - C: Path Adherence
 - D: Overall Ranking
- 9. CONCLUSION & FUTURE WORK**

1. INTRODUCTION — The Local Navigation Task

local controller chooses one trajectory *every cycle* to follow the **global path** safely — trading off progress, clearance and smoothness.

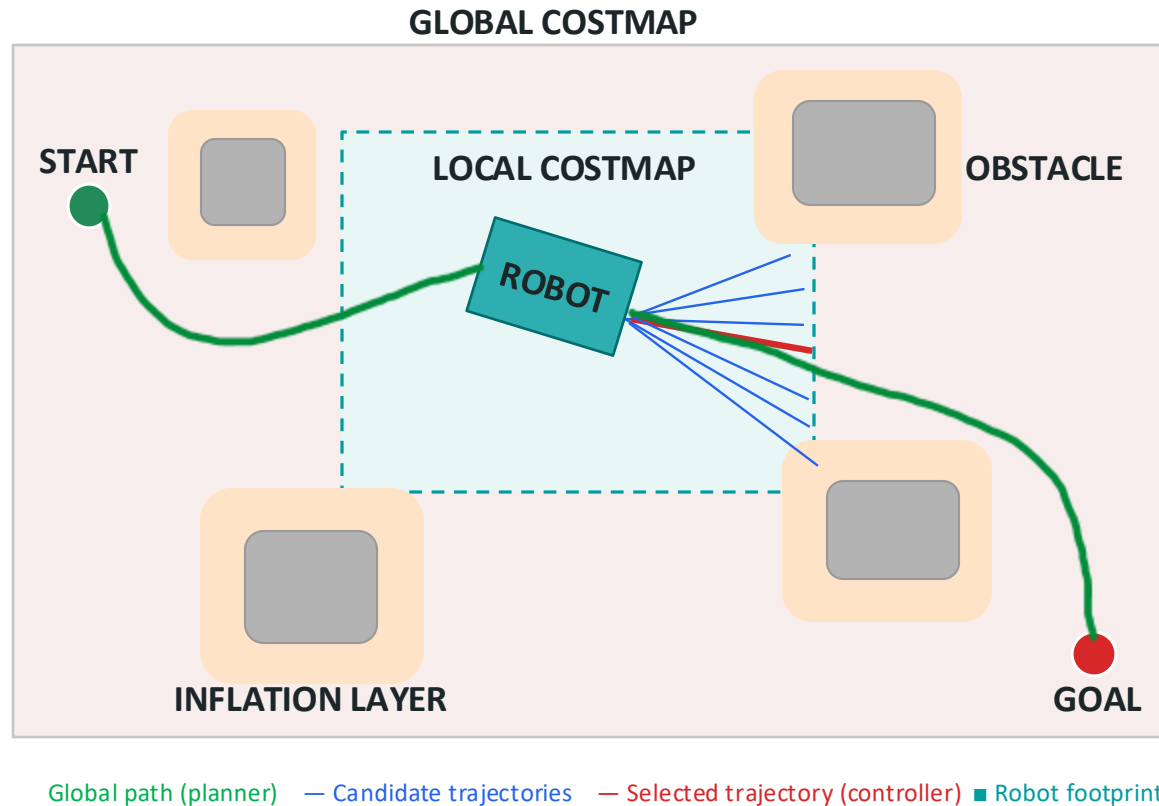


Figure 1 (A): Illustrative diagram for Robot Navigation



Figure 1 (B): Illustrative diagram for Costmaps and Inflation

Global planner provides path inside global costmap from Start to Goal (Green), Local controller samples candidates (blue) inside its rolling local costmap and runs the selected one (red).

2. LITERATURE REVIEW — Classical vs Learned vs Hybrid

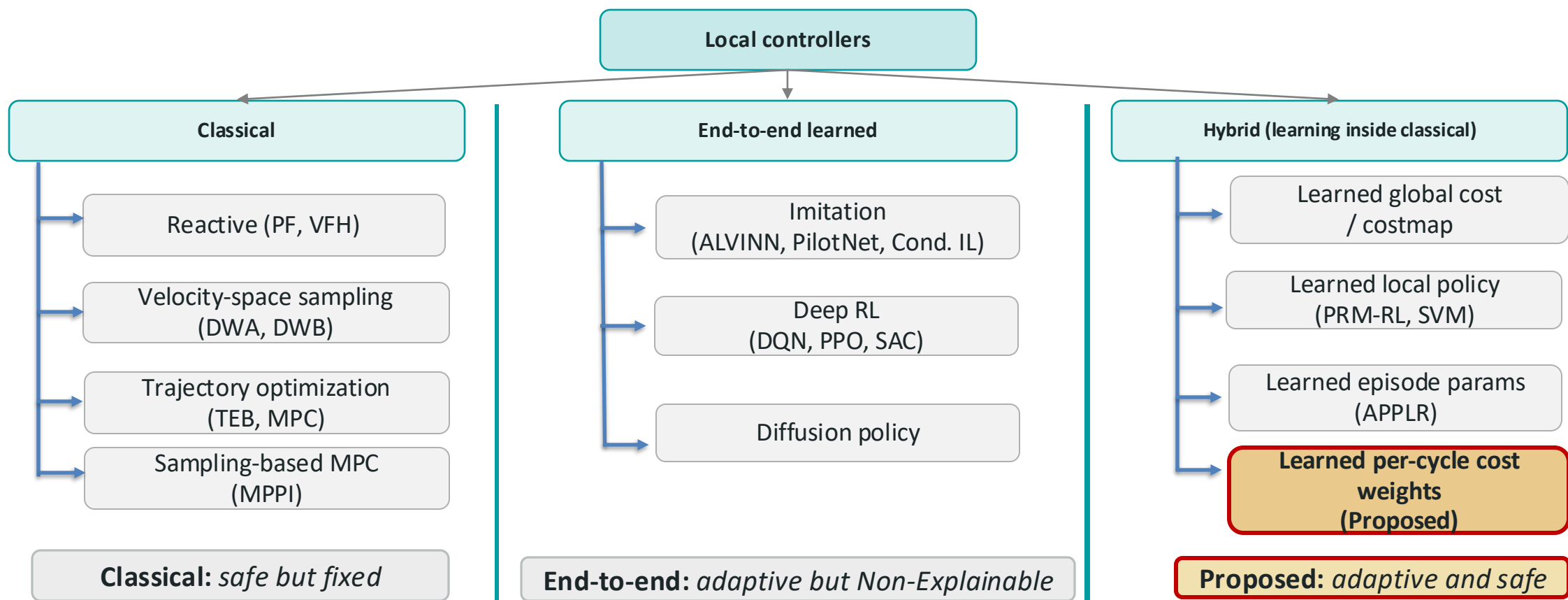


Figure 2 — Taxonomy of local controllers.

Gap: adapt to context *every cycle* while keeping the hard safety of the classical design.

Classical: Khatib 1986; Borenstein & Koren 1991; Fox et al. 1997 (DWA); Rösmann et al. 2013 (TEB); Williams et al. 2017 (MPPI); Macenski et al. 2023.

End-to-end / learned: Bojarski et al. 2016 (PilotNet); Codevilla et al. 2018 (Cond. IL); Chi et al. 2023 (Diffusion); Schulman et al. 2017 (PPO); Haarnoja et al. 2018 (SAC).

Hybrid: Faust et al. 2018 (PRM-RL); Charalampous et al. 2015 (SVM); Xu et al. 2021 (APPLR); Wang et al. 2021 (survey).

3. OBJECTIVES — Objectives & Research Questions

Objective

- Real-time controller : a small network sets per-cycle cost weights.
- Keeps classical modularity, interpretability and hard safety.
- Learning touches only the weighting, not the planner.
- Goal: beat fixed-weight and end-to-end baselines.

Research Questions

- **RQ1 (Feasibility)**. Learn per-cycle weight multipliers within the real-time (~50 ms) budget?
- **RQ2 (vs. fixed-weight)**. A better multi-criteria balance (safety, efficiency, motion, path) without losing success?
- **RQ3 (Structural safety)**. Is the hard guarantee preserved by confining learning to the weighting?
- **RQ4 (vs. end-to-end IL)**. Learned weights better & safer than directly-learned velocities (data fixed)?
- **RQ5 (Taxonomy & signature)**. Interpretable Reynolds grouping, with a context-adaptation signature in the weights?
- **RQ6 (Training paradigm)**. Raw recorded costs vs. oracle-recovered weights — which wins?

4. METHODOLOGY

A — Overview: Safety by Decomposition

Separating what is learned from what is controlled.

A network emits a per-cycle multiplier m_t that rescales the configured cost weights.

Safety by decomposition.

Learning touches only the weighting; the planner keeps feasibility, obstacle handling & velocity generation.

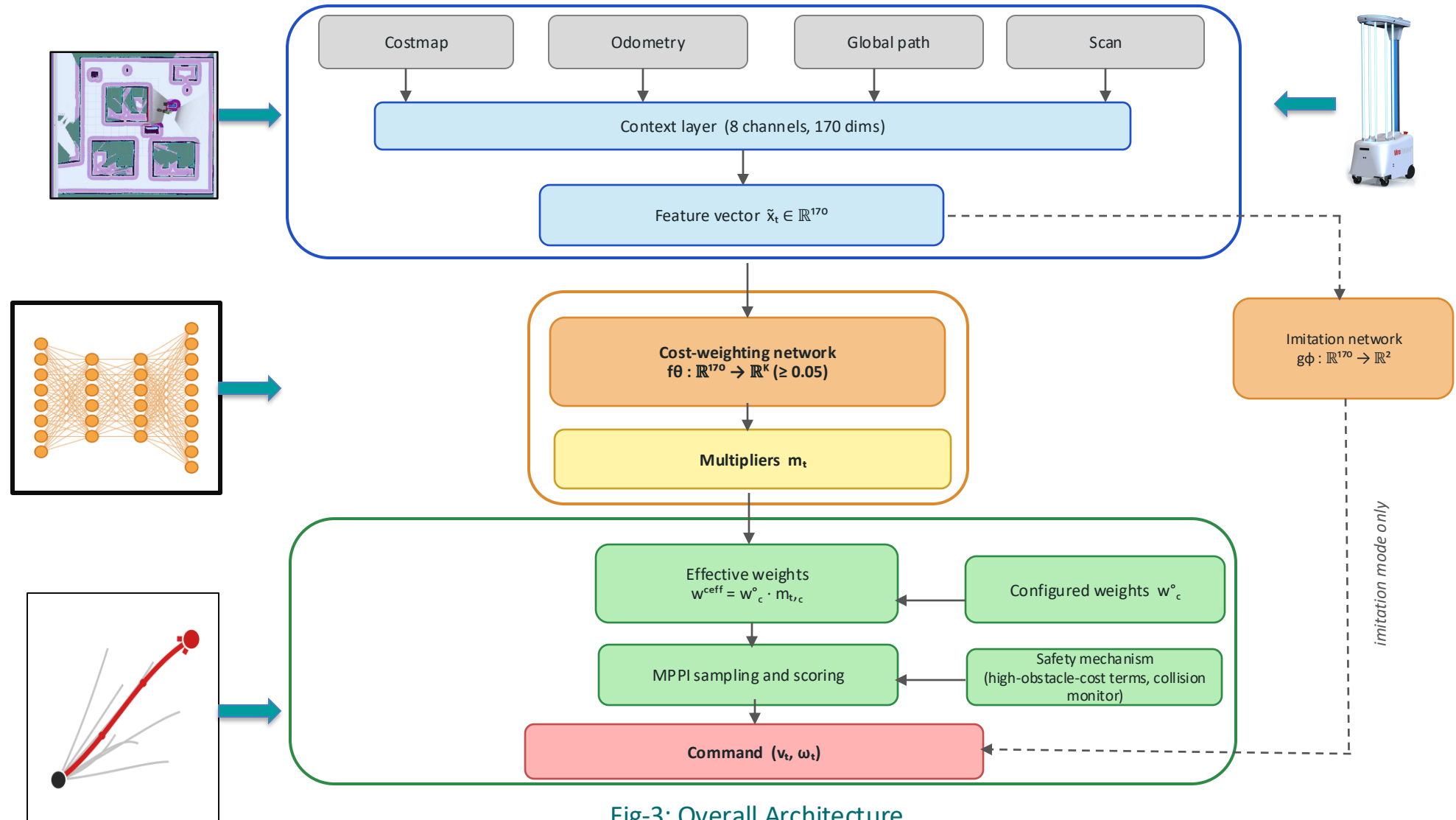


Fig-3: Overall Architecture.

4. METHODOLOGY

B — Perception Input (170-D Feature Vector)

A single 170-dimensional vector, partitioned into eight named channels (three provenance families), conditions the network each cycle.

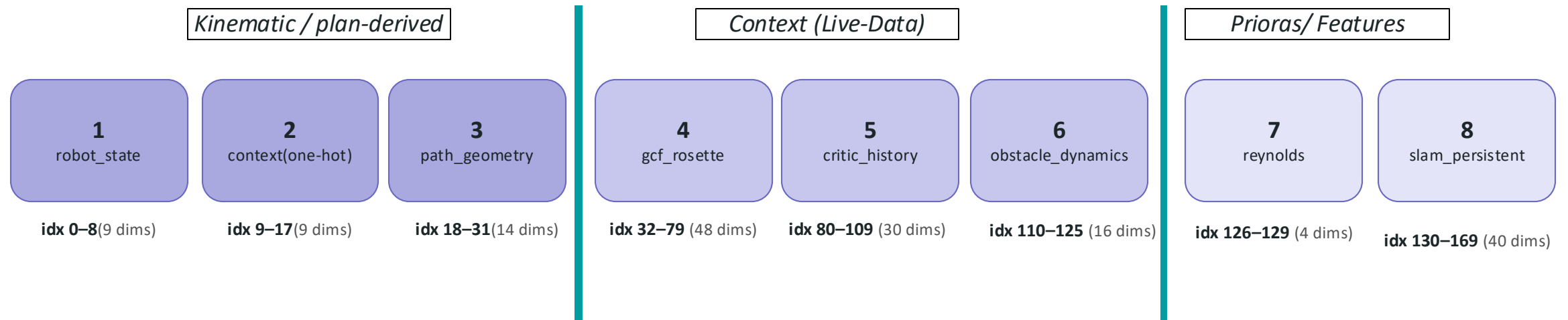


Fig-4: Index-range layout of the v3 perception vector x_t .

Context one-hot (channel 2, indices 9-17):
9: open 10: corridor 11: doorway 12: junction
13: cluttered 14: approaching-goal
15-17: reserved (zero)

Input versions:
v1 = channels 1-6 (126 dims)
v2 = v1 + channel 7 (130 dims)
v3 = v2 + channel 8 (170 dims, headline)

4. METHODOLOGY

C — Cost-Weighting Network

A channel-wise frontend feeds a (256, 128, 64) fusion stack and a paradigm-specific head: $64 \rightarrow 11 + \text{softplus}$ for the multiplier paradigms, $64 \rightarrow 2$ for the imitation baseline.

Input:

Per-channel:

Fusion:

Output:

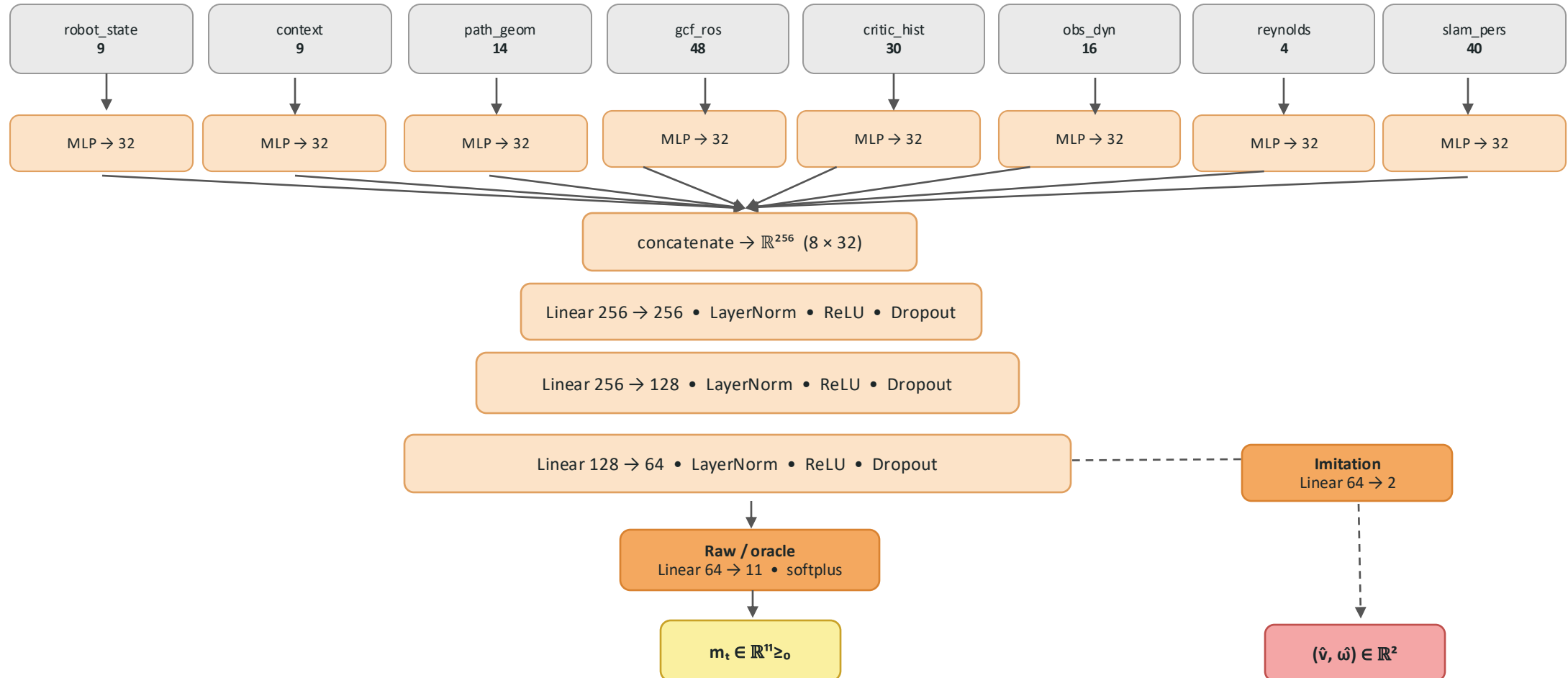


Fig-5: End-to-end structure of the learned controllers.

4. METHODOLOGY

D — Weighted MPPI Rollout

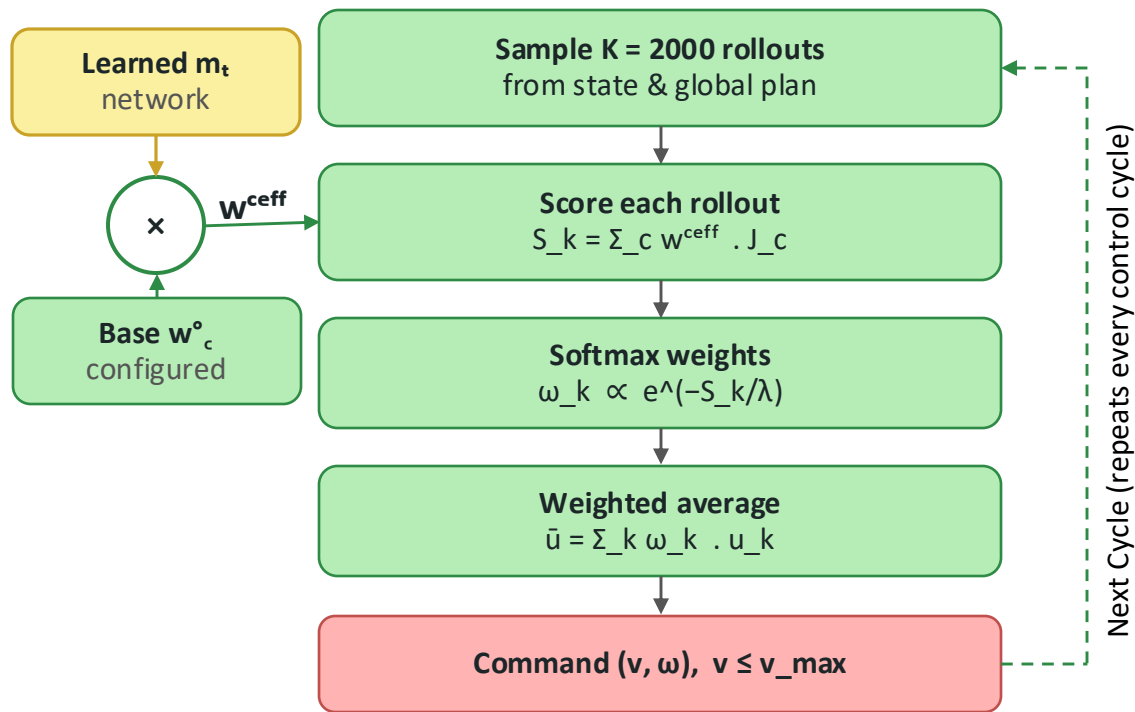


Fig-6 (A): MPPI control Cycle (Classical)

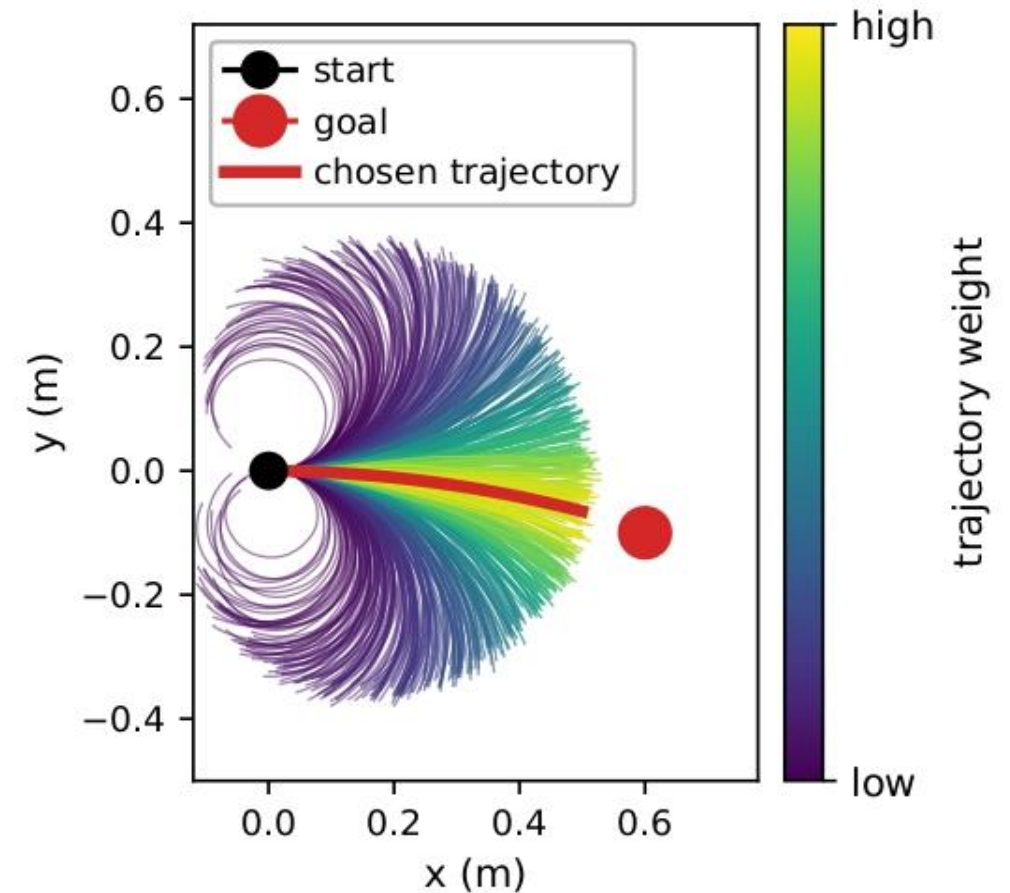


Fig-6 (B): Trajectory generation and selection (illustrative.)
2000 rollouts, one 2.0 s horizon. The weights bias which rollouts win the SoftMax.

4. METHODOLOGY

E — Control Algorithm & Critic Taxonomy

Algorithm 1 — CA-MCW control cycle

```
1:  $x_t \leftarrow \text{assembleFeatures}(\text{costmap}, \text{odom}, \text{path}, \text{scan})$ 
2:  $m_t \leftarrow \text{weightProvider}(x_t)$ 
3: for cost term  $c \in \{1, \dots, K\}$  do
4:    $w^{\text{eff}} \leftarrow w_0 \cdot m_t,^c$ 
5: end for
6:  $\tau^* \leftarrow \text{MPPI}(\{w^{\text{eff}}\})$ 
7:  $(v_t, \omega_t) \leftarrow \tau^*[0]$ 
8: return  $(v_t, \omega_t)$ 
```

The multiplier enters only the cost-weight product; feasibility and command extraction stay with the planner.

Reynolds-grouped critics.

The ten cost terms map to named, auditable behaviors from Reynolds' flocking primitives, adapted to single-robot navigation:

Primitive	Role
-----------	------

Separation	— clear of obstacles / walls
------------	------------------------------

Alignment	— match heading to plan
-----------	-------------------------

Cohesion	— stay on the global path
----------	---------------------------

Goal-seeking	— progress to the goal
--------------	------------------------

Gives the learned weight vector a physically meaningful axis and a basis for group-level ablation.

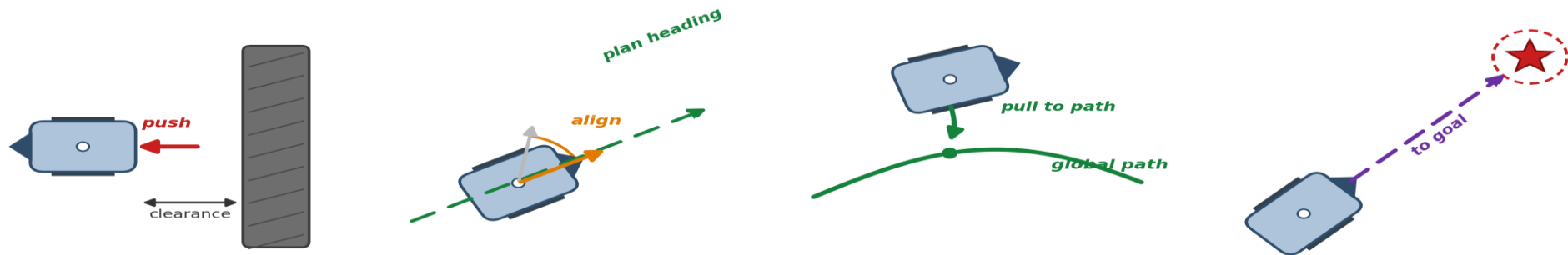


Fig-7: Reynolds-grouped critics (Illustrative)

5. EXPERIMENTAL DESIGN

Simulation — Training



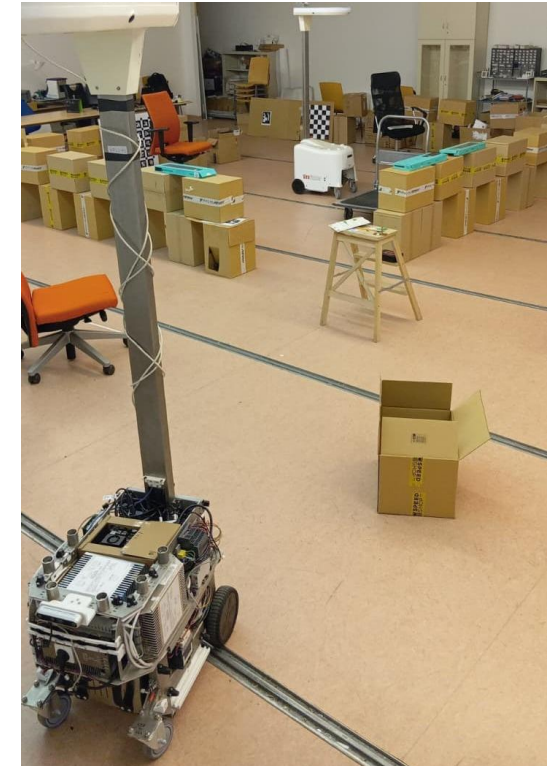
Env 1: wide, open
Env 2: narrow corridors
Env 3: dense clutter

Simulation — Deployment (unseen)



Balanced — open space, corridors & clutter

Real-world — Deployment



Cluttered, hospital-like — robot, chairs, trolley, gear

Fig-8: Experimental Design Environments

6. TRAINING CONVERGENCE

One representative model per paradigm — train (solid) vs. validation (dashed); early stopping fires before overfitting.

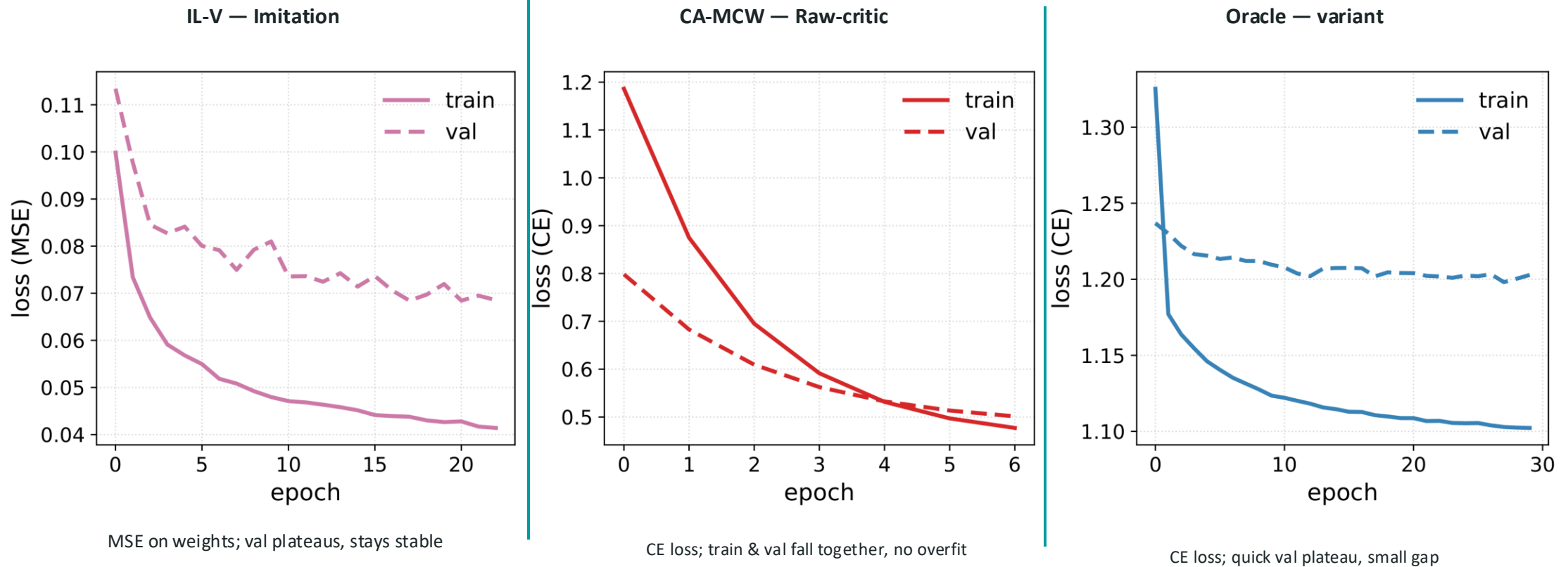


Fig-9: Training Convergence Loss vs Epoch Curves

7. EVALUATION METRICS

Per-episode metrics grouped into six categories (five scored, one diagnostic). Winner Selected by TOPSIS method

Outcome

- success rate
- timeout / abort / rescue
- goal position & heading error

Motion quality

- mean / max jerk
- angular-velocity std
- command accel. RMS
- command sign-flips

Efficiency

- SPL, safe-SPL
- duration
- actual path length
- linear velocity, stall fraction

Path adherence

- mean / median / RMSE XTE
- max / p95 XTE
- path coverage

Safety

- collision
- min & p5 clearance
- min distance to obstacle
- near-miss rate

Adaptation (diagnostic)

- critic entropy (normalized)
- total variation, TV / s
- dominant-critic fraction

TOPSIS Formulation

$$C_i^* = \frac{S_i^-}{S_i^+ + S_i^-} \in [0, 1]$$

$$S_i^+ = \sqrt{\sum_j (v_{ij} - v_j^+)^2}, \quad S_i^- = \sqrt{\sum_j (v_{ij} - v_j^-)^2}$$

where

C_i^* closeness of controller i to the ideal (rank by it, top-1 wins)

S_i^+, S_i^- distance to the best / worst ideal point

v_{ij} weighted, normalised score of controller i on criterion j

v_j^+, v_j^- best / worst value in column j ($\max_i, \min_i$)

8. RESULTS

A — Learned Adaptation Signature

Per-critic weight share varies with local environment : a **context-conditioned schedule**, not a constant vector. It learns its own effective schedule.

Learned context schedule: how CA-MCW re-weights each cost as obstacles get closer

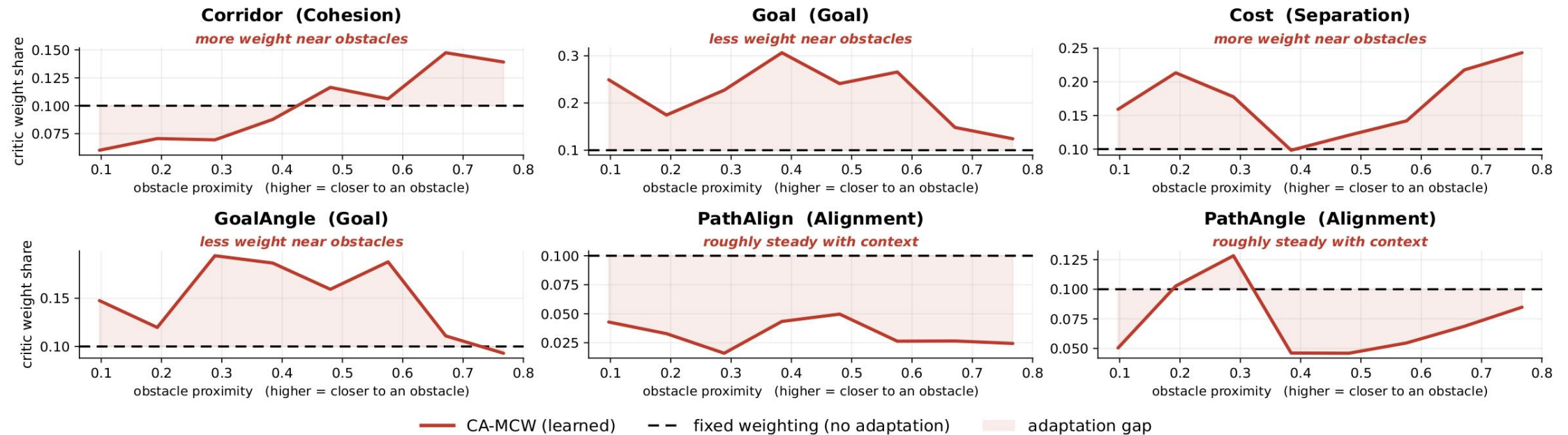


Fig-10: Learned Adaptation Signature

8. RESULTS

B— Path Adherence

Cross-track error against each controller's own active plan: the five completing controllers track tightly (~ 0.02 m); IL-V is the outlier.

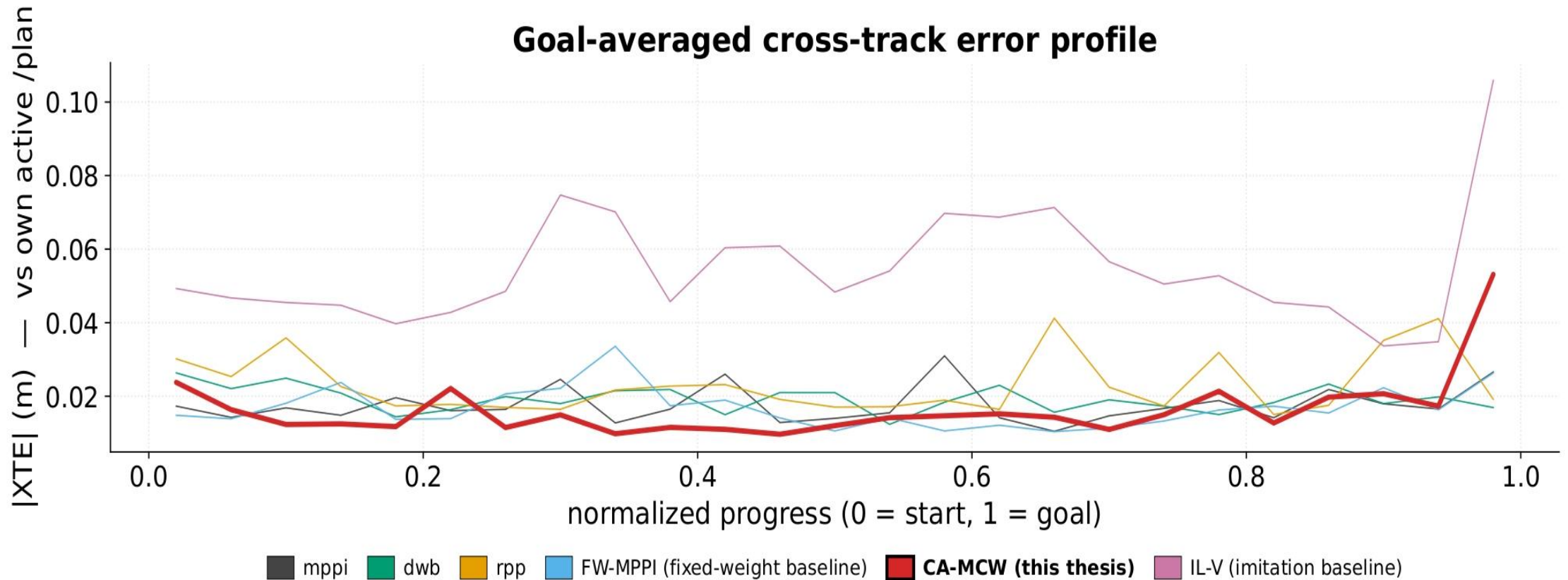


Fig-11: Path Adherence (Result)

8. RESULTS

C— Tour & Overall Ranking: (my CA-MCW Ranks First)

CA-MCW (Proposed) outranks every classical baseline, imitation-variant and fixed-weight variant

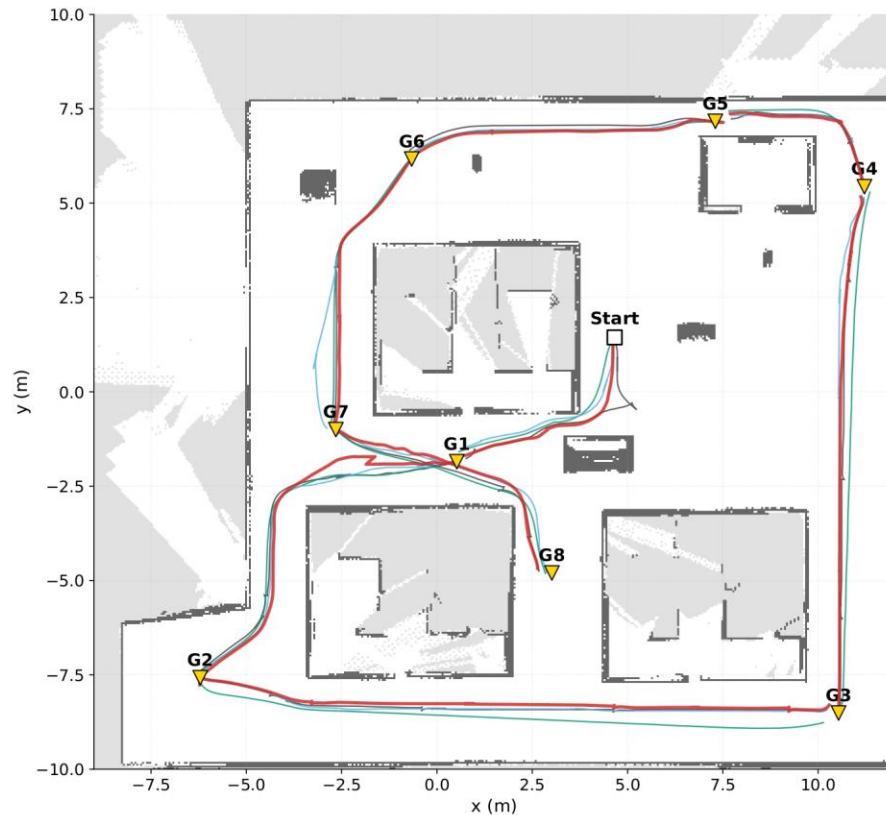


Fig-12 (A): Eight-goal evaluation tour (fully-successful controllers).

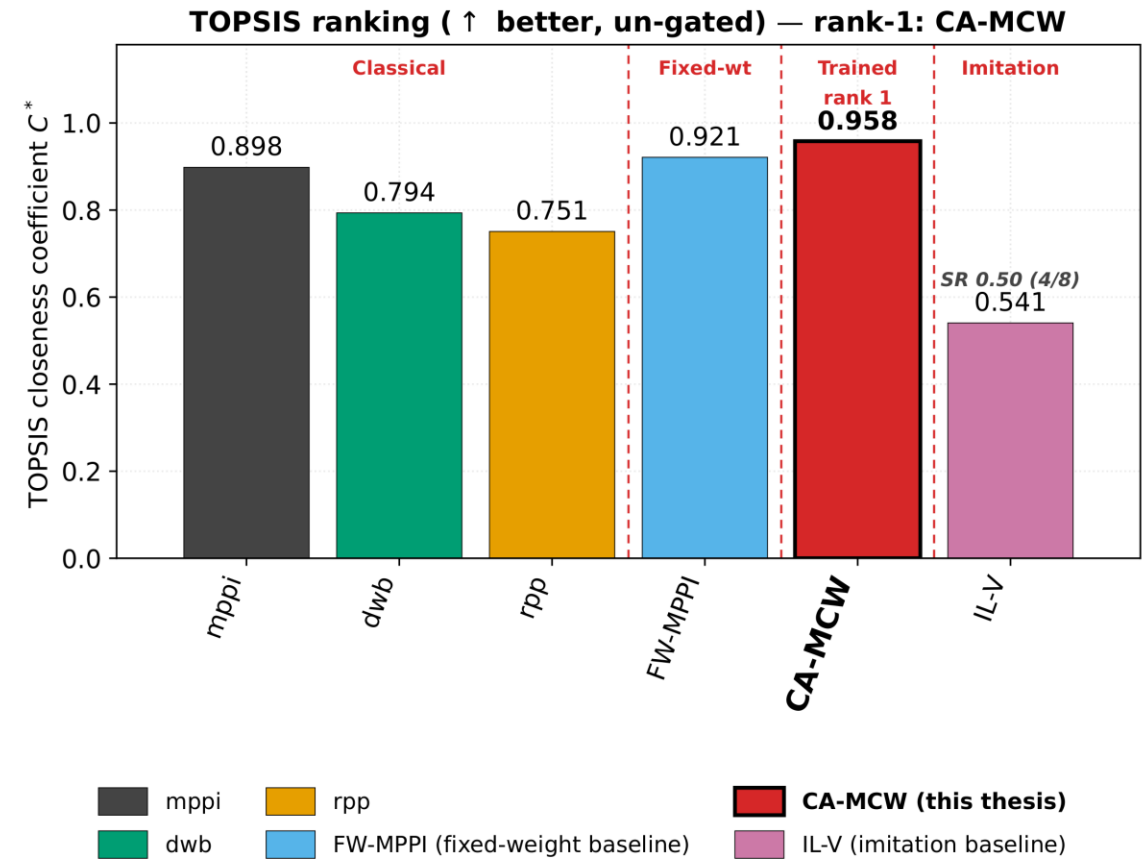
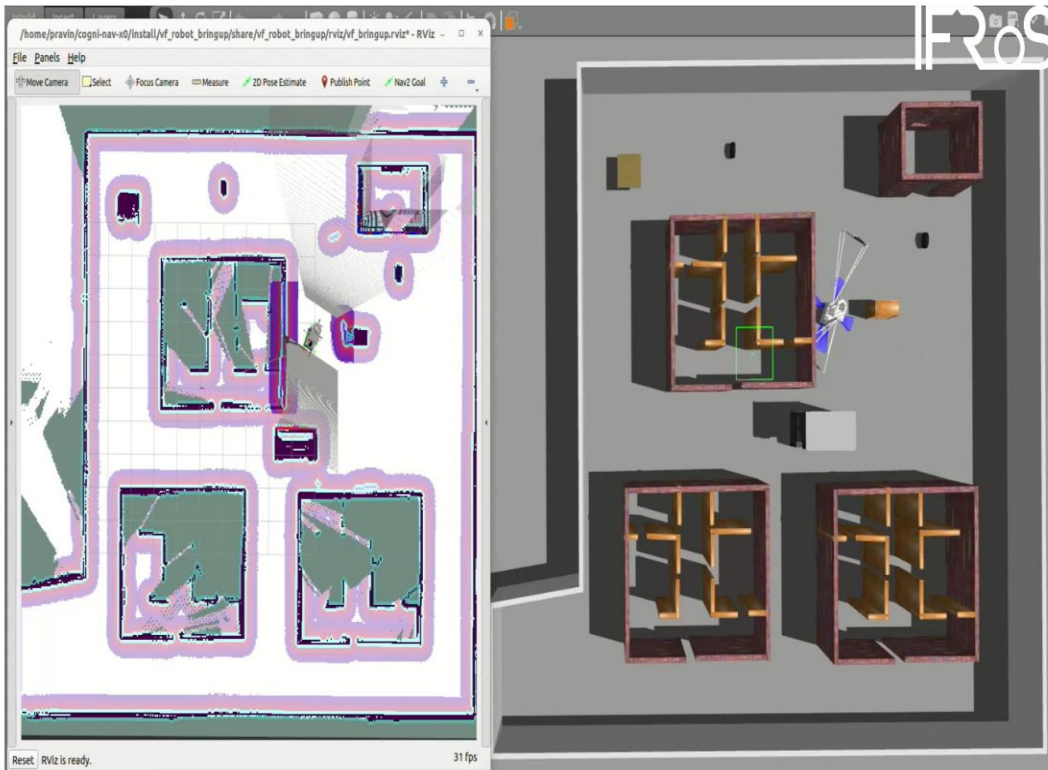


Fig-12 (B): TOPSIS closeness C^* (higher is better); IL-V shown un-gated.

8. RESULTS

D— Demonstration Video

Simulation Environment (Gazebo)



Real Environment (Laboratory workspace)



9. CONCLUSION & FUTURE WORK

Conclusion:

1. By **splitting the local planner (controller)** into a **learned per-cycle cost weighting** and a **classical safety-bearing formulation** lets my Context Adaptive Multi Cost Weighting (**CA-MCW**) controller, **adapt to context without surrendering the hard guarantees of sampling-based control**.
2. It **outranks every classical baseline, the fixed-weight reference and my own imitation weight reference** variants on the multi-criteria aggregate, and has been successfully deployed on the physical robot, not only in simulation.

Future Work:

- a. **Dynamic scenes:** moving obstacles and crowds beyond the static tour.
- b. **Joint learning:** train base weights wc° and multipliers mt end-to-end rather than rescaling a fixed configuration.
- c. **Broader real-world trials:** longer missions and a quantitative sim-to-real gap study.
- d. **Scaling:** larger critic banks and richer context channels.

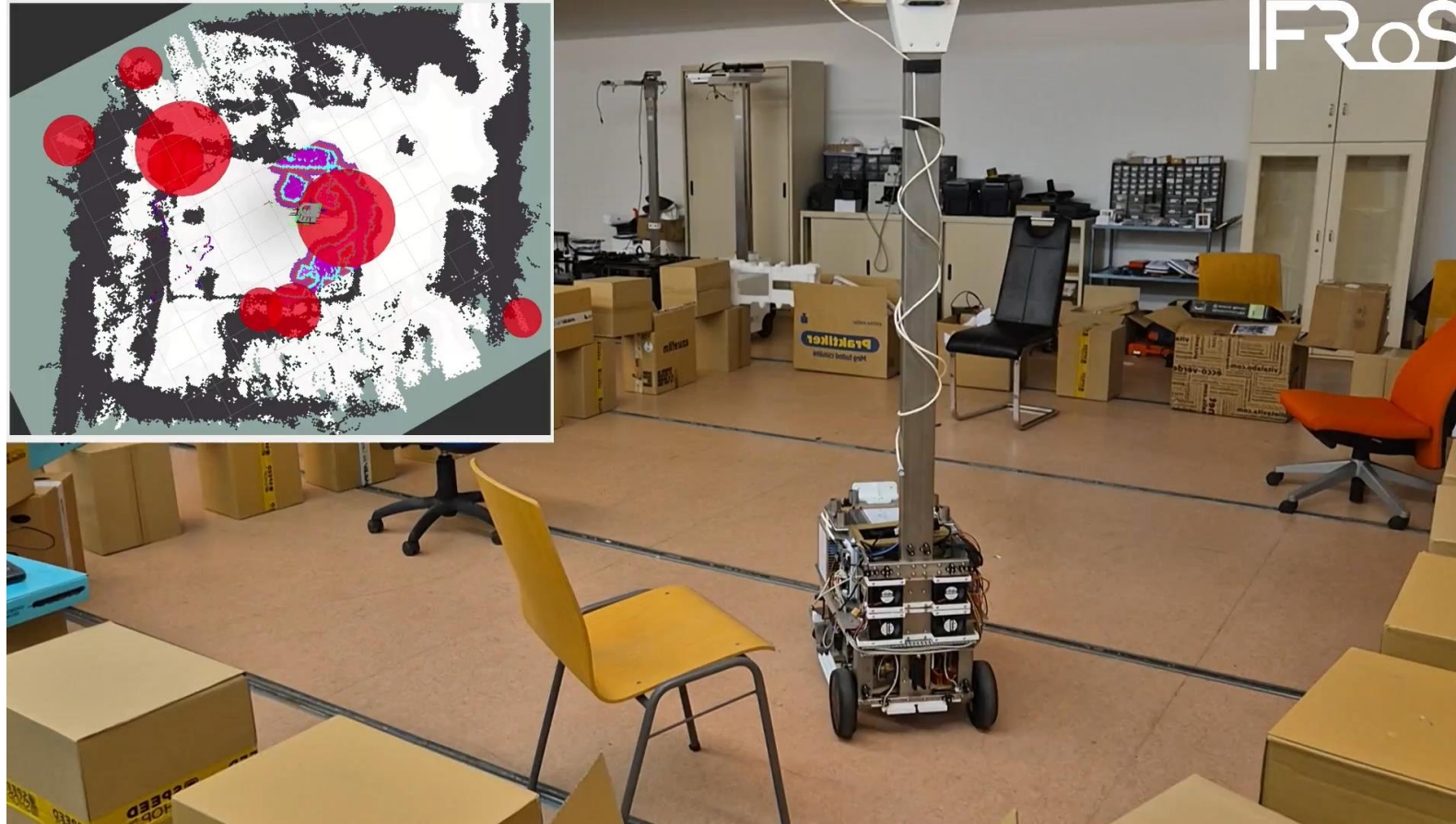
Thesis Summary:

Separating *what is learned* (weighting) from *what stays classical* (safety) yields a controller that is both context-adaptive *and* safe by construction — the best-balanced of all baselines and own reference variants evaluated.

10. Bonus

Autonomous Disinfection Video

Autonomous Disinfection Demo (Laboratory workspace)



Thank You

Questions & Discussion



LinkedIn



YouTube Video

Pravin Oli | pravin.oli.08@gmail.com

Special Thanks to My Supervisors

Internal supervisor: **Prof. Zoltán Istenes** (ELTE)

External supervisors: **R. Tommasi & Dr. D. Horváth** (European Knowledge Centre Ltd.)



ELTE
EÖTVÖS LORÁND
UNIVERSITY

